

7/22/23, 3:13 PM

CodeProjectFileEditFindViewToolsEducationHelpNode versionProject index (opens)Configure...

temp.py x Terminal

1 # WRITE YOUR CODE HERE

2 import os

3 import openai

4 import secret

5 openai.api_key='sk-QVkkVGvk7ose2G3HCQh7T3B1bkFJi8vDKIMdS8Rk6rTsg1Y'

6

7 response=openai.ChatCompletion.create(

8 model="gpt-3.5-turbo",

9 messages=[

10 {"role": "system", "content": "You are a financial advisor."},

11 {"role": "user", "content": "What are some tips for saving money?"},

12 {"role": "assistant", "content": "Creating a budget, reducing expenses, and saving on utilities"},

13 {"role": "user", "content": "How do I create a budget?"}

14]

15)

16

17 print(response['choices'][0]['message']['content'])

18

19 response=openai.ChatCompletion.create(

20 model="gpt-3.5-turbo",

21 messages=[

22 {"role": "system", "content": "You are a travel expert specializing in European destinations"},

23 {"role": "user", "content": "Give me the top 2 tourist destinations in Germany for my honeymoon"}]

24

25)

Refining System Messages for Effectiveness

System messages play a crucial role in guiding the AI assistant's behavior when interacting with the ChatGPT API. In this guide, we will explore the importance of refining system messages, techniques for crafting effective instructions, and examples to help you improve the quality of generated responses.

The system message sets the context and role for the AI assistant. It provides initial instructions on how the assistant should behave throughout the conversation. By carefully crafting system messages, you can ensure the AI understands the desired behavior, leading to more accurate and relevant responses.

Techniques for Crafting Effective System Messages:

- Be specific:** Clearly define the assistant's role or expertise to set proper expectations for the conversation.
- Set constraints:** If necessary, provide limitations or rules that the assistant should follow when generating responses.
- Offer context:** Mention any relevant background information that could influence the assistant's understanding or responses.

Lets try a few examples, First in order to interact with the API we need to get our libraries and our API key.

```
import os
import openai
import secret
openai.api_key=secret.api_key
```

TRY IT

Command was successfully executed.

Now we are ready to use our API call which we will set equal to `response` so its easier for us to manipulate.

```
response=openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "system", "content": "You are a financial advisor."},
        {"role": "user", "content": "What are some tips for saving money?"},
        {"role": "assistant", "content": "Creating a budget, reducing expenses, and"},
        {"role": "user", "content": "How do I create a budget?"}
    ]
)
```

TRY IT

Terminated by timeout

We are going to use some slicing in order to make our response more legible.

```
print(response['choices'][0]['message']['content'])
```

TRY IT

Terminated by timeout

In the above example we are setting the AI as financial advisor. It provides initial instructions on how the assistant should behave throughout the conversation.

However, we can use more precision, with our tasks. Lets replace our message with the following.

```
{"role": "system", "content": "You are a travel expert specializing in European destinations"}
```

TRY IT

Explanation: This system message clearly defines the assistant's role as a travel expert and narrows down the expertise to European destinations, providing more focused guidance. Now if the user want to add something more specific, they can add it to the conversation.

ChatGPT API

Which of the following would not generate an error?



```
response=openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "system", "content": "You are a travel expert"},
        {"role": "user", "content": "Give me the top 2 tourist destinations in Germany"}
    ]
)
```



```
response=openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "system", "content": "You are a travel expert"},
        {"role": "user", "content": "Give me the top 2 tourist destinations in Germany"}
    ]
)
```

In the second code block, would generate an error as we can see that because the closing brackets are in the wrong place:

```
messages=[
    {"role": "system", "content": "You are a travel expert specializing in European destinations"},
    {"role": "user", "content": "Give me the top 2 tourist destinations in Germany"}
]
```

Check It

Next